About resmon Page — Info Document

Page Overview

Purpose

The About resmon page is the in-app documentation and identity surface for `resmon`. It hosts four self-contained sub-tabs — **Tutorials**, **Issues**, **Blog**, and **About App** — under a single top-level route. The Tutorials tab hosts a guided walk-through for every existing top-level page (Dashboard, Deep Dive, Deep Sweep, Routines, Calendar, Results & Logs, Configurations, Monitor, Repositories & API Keys, Settings) and every Settings sub-tab (Email, Cloud Account, Cloud Storage, AI, Storage, Notifications, Advanced), plus a full-app overview entry. The Issues tab provides a credentials-free path to file bug reports, feature requests, and questions either by email or as a pre-populated GitHub issue. The Blog tab embeds the public resmon GitHub Pages blog so per-update posts can be read in-app without a release cycle. The About App tab — relocated out of Settings in this update — surfaces the build version, recent-update notes, license, privacy notice, and author social links.

The page also defines the shared `TutorialLinkButton` component that is rendered next to every page header (and next to each Settings sub-panel header) across the app, so any user can jump from an arbitrary page directly to that page's tutorial section in a single click.

Primary User Flows

1. Open **About resmon** from the sidebar. The page renders an `<h1>About resmon</h1>` header and a tab strip with four tabs (Tutorials → Issues → Blog → About App); the route defaults to the Tutorials tab.
2. Click **Tutorials** (`/about-resmon/tutorials`) to read the table of contents and any of the eighteen tutorial sections (one full-app overview, ten page sections, and seven Settings sub-tab sections). Click a TOC entry, a tutorial **Tutorial** button on any page, a section's **Go to Page** / **Go to Tab** button, or the section-level **prev** / **next** buttons to navigate within the tab.
3. Click **Issues** (`/about-resmon/issues`) to fill out the issue-report form (issue type, title, description, steps-to-reproduce, expected-vs-actual, optional contact email) and use either **Open in Email** (pre-populated `mailto:` link) or **File on GitHub** (pre-populated GitHub issue deep link) to submit. The app itself never posts the report; the user reviews and sends in their email client or on GitHub.
4. Click **Blog** (`/about-resmon/blog`) to browse the public resmon blog. The tab fetches the GitHub Pages Atom feed and renders a two-pane layout (left: post list; right: an Electron `<webview>` showing the selected post). Off-origin links open in the user's default browser.
5. Click **About App** (`/about-resmon/about-app`) to view the version banner, recent-update card, license card, privacy notice card, and author card.
6. From any page or Settings sub-panel header, click the **Tutorial** button rendered by `TutorialLinkButton` to deep-link straight to the matching tutorial anchor (the URL hash drives a smooth-scroll into view).

Inputs and Outputs

- **Inputs** — `GET /api/health` (About App tab, to display the live backend version); URL hash on the Tutorials route (drives smooth-scroll into the matching `#tutorial-<anchor>` section); the Atom feed fetched by the Blog tab from `https://ryanjosephkamp.github.io/resmon/feed.xml` ; the user-typed form fields on the Issues tab.
- **Outputs** — no server-side writes are produced from this page. The Issues tab routes to the user's default mail client (`mailto:` URL via `shell.openExternal`) or to GitHub's new-issue page (`https://github.com/ryanjosephkamp/resmon/issues/new?...` via `shell.openExternal`), but the app itself never transmits the report. All other controls are navigational or read-only.

Known Constraints

- The page is read-only: no mutating backend endpoints are wired to any of the four tabs. The Issues tab delegates submission to the user's email client or to GitHub; no credentials are stored or transmitted by `resmon` itself.
- The Blog tab requires network access to `https://ryanjosephkamp.github.io` to fetch the feed and render posts; on feed-fetch failure it falls back to a single-pane embed of the blog index plus an "Open in browser" button.
- Tutorial media are intentionally placeholder cards (one YouTube placeholder for the full-app overview and one GIF placeholder per page / sub-tab); the scaffolding lets every page deep-link land on a stable anchor today and the actual media may be added later without re-routing.
- The About App tab seeds its version display to `1.0.0` and refreshes from `GET /api/health.version` (sourced from `implementation_scripts.config.APP_VERSION`); when the daemon health endpoint is unreachable, the seed value is preserved.
- The author profile photo (`assets/kamp_profile_pic.png`) is bundled with the renderer; no network fetch is required to render the author card.

Frontend

Route and Main Component

- **Route:** `/about-resmon` (nested routes under `/about-resmon/<tab>`).
- **Main component:** `AboutResmonPage` (resmon_scripts/frontend/src/pages/AboutResmonPage.tsx).
- The page renders an `<h1>About resmon</h1>` header inside a `.page-header` block, a `.settings-nav` tab strip of four `NavLink` elements (Tutorials → Issues → Blog → About App), and a nested `<Routes>` block. The index route redirects to `/about-resmon/tutorials` via `<Navigate to="tutorials" replace />` .

Child Components and Hooks

- Tab components (under `resmon_scripts/frontend/src/components/AboutResmon/`):
  - `TutorialsTab` (resmon_scripts/frontend/src/components/AboutResmon/TutorialsTab.tsx).
  - `IssuesTab` (resmon_scripts/frontend/src/components/AboutResmon/IssuesTab.tsx).
  - `BlogTab` (resmon_scripts/frontend/src/components/AboutResmon/BlogTab.tsx).
  - `AboutAppTab` (resmon_scripts/frontend/src/components/AboutResmon/AboutAppTab.tsx).
- Shared deep-link component:
  - `TutorialLinkButton` (resmon_scripts/frontend/src/components/AboutResmon/TutorialLinkButton.tsx) — a small primary-styled button rendered next to every page header (Dashboard, Deep Dive, Deep Sweep, Routines, Calendar, Results & Logs, Configurations, Monitor, Repositories & API Keys, Settings) and next to each Settings sub-panel header (Email, Cloud Account, Cloud Storage, AI, Storage, Notifications, Advanced). Clicking the button navigates to `/about-resmon/tutorials` with `hash: <anchor>` .
- Reused shared components: `PageHelp` (storage key `about-resmon-about-app`).
- Hooks: `NavLink` , `Navigate` , `Routes` , `Route` , `useNavigate` , `useLocation` from `react-router-dom` ; `useEffect` , `useRef` , `useState` for tab-local state; `apiClient` for the version `GET` .

UI State Model

`AboutResmonPage` itself holds no state. Each tab owns its own local state.

- **TutorialsTab** state:
  - A static `sections: TutorialSection[]` array that drives the TOC and the rendered sections (one full-app entry plus seventeen destination-bearing entries: ten pages and seven Settings sub-tabs).
  - A `containerRef` for the panel root.
  - `useLocation().hash` is observed via `useEffect` ; when it changes, the matching `#tutorial-<anchor>` element is scrolled into view via `scrollIntoView({ behavior: 'smooth', block: 'start' })` .
- **AboutAppTab** state:

- `backendVersion: string` — seeded to `'1.0.0'`, refreshed from `GET /api/health` on mount.
- **TutorialLinkButton** state — none; the component is purely a click-to-navigate wrapper around `useNavigate`.

Key Interactions and Events

- **Tab switch** — the two `NavLink` elements update the URL; the matching tab mounts and (in the case of About App) issues a single health `GET`.
- **Tutorial deep-link** — every `TutorialLinkButton` calls `navigate({ pathname: '/about-resmon/tutorials', hash: anchor })`. On arrival, the Tutorials tab's hash effect scrolls the matching section into view.
- **TOC click** — every TOC entry calls `goToAnchor(s.anchor)`, which performs the same hash-only navigation (no full re-render).
- **Section nav** — each section's `Go to Page` / `Go to Tab` button calls `navigate(s.destination.path)` and routes away from the About resmon page; the **prev** / **next** buttons under each section call `goToAnchor` to traverse the section list in declared order.

Error and Empty States

- The page itself does not render a loading skeleton; both tabs render synchronously.
- `AboutAppTab` swallows version-fetch errors silently and keeps the seeded fallback (`'1.0.0'` initial; the version card uses the resolved value or falls back to `'1.1.0'` when the state value is empty) so the panel never shows an error chrome.
- The Tutorials tab has no fetch path and therefore no error state; if the URL hash matches no anchor, the smooth-scroll effect simply no-ops.

Tab: Tutorials

- Component: `TutorialsTab` (resmon_scripts/frontend/src/components/AboutResmon/TutorialsTab.tsx).
- Renders `<h2>Tutorials</h2>`, an introductory paragraph, a `.tutorial-toc` table-of-contents block, and one `<section className="tutorial-section">` per entry in the static `sections` array.
- Each section emits the following structure:
  - A `.tutorial-section-header` flex row containing an `<h3>` title (`id="tutorial-<anchor>-title"`) and, when `destination` is set, a primary-styled `Go to Page` or `Go to Tab` button. The full-app overview entry has no `destination` and therefore no destination button.
  - A short blurb paragraph.
  - A `.tutorial-media-placeholder` card with the `mediaCaption` text (one YouTube placeholder for full-app, one GIF placeholder per page or sub-tab).
  - A `.tutorial-details` block with three sub-blocks: **How to use it** (ordered list of `instructions`), **Special features** (unordered list of `features`), and **Tips & tricks** (unordered list of `tips`).
  - A `.tutorial-nav` row with **prev** and **next** buttons that traverse the declared section order; the first section omits prev and the last section omits next (each replaced by an empty `<span />` placeholder for grid alignment).
- Section anchors (in declared order): `full-app`, `dashboard`, `deep-dive`, `deep-sweep`, `routines`, `calendar`, `results`, `configurations`, `monitor`, `repositories`, `settings`, `settings-email`, `settings-account`, `settings-cloud`, `settings-ai`, `settings-storage`, `settings-notifications`, `settings-advanced`.
- Anchor synchronization is one-way (URL → scroll). The component does not write the URL hash on scroll; it only reads it on each `location.hash` change.

Tab: Issues

- Component: `IssuesTab` (resmon_scripts/frontend/src/components/AboutResmon/IssuesTab.tsx).
- Renders an `<h2>Issues</h2>` heading, a brief intro explaining the credentials-free reporting path, and a single form with the following fields:
  - **Issue type** radio selector — `Bug`, `Feature request`, `Question`, `Other`.
  - **Title** text input.
  - **Description** textarea.
  - **Steps to reproduce** textarea.

    - **Expected vs. actual** textarea.
    - **Optional contact email** input.
- The tab never POSTs anything itself. It exposes two read-only submit paths:
    i. **Open in Email** — builds a `mailto:ryanjosephkamp@gmail.com?subject=...&body=...` URL pre-filled with the form contents and an auto-collected diagnostic block (app version from `GET /api/health`, current route hash, user-agent, platform). The user reviews and clicks Send in their default mail client.
    ii. **File on GitHub** — builds a `https://github.com/ryanjosephkamp/resmon/issues/new?template=...&labels=...&title=...&body=...` URL whose query params line up with the typed GitHub issue forms under `.github/ISSUE_TEMPLATE/` so the matching form (`bug.yml`, `feature.yml`, `question.yml`) auto-selects on the GitHub side.
- Both buttons go through `window.resmonAPI.openPath`, which routes `mailto:` and `https:` URLs through `shell.openExternal` in the Electron main process; falls back to `window.location.href` (mailto) and `window.open(url, '_blank')` (https) when the preload bridge is unavailable.
- No credentials are stored, cached, or transmitted by `resmon` itself; submission is fully delegated to the user's email client or the GitHub web UI.

Tab: Blog

- Component: `BlogTab` (resmon_scripts/frontend/src/components/AboutResmon/BlogTab.tsx).
- Fetches the GitHub Pages Atom feed at `https://ryanjosephkamp.github.io/resmon/feed.xml`, parses it client-side via `DOMParser` into a typed `BlogPost[]`, and renders a two-pane layout:
    - **Left pane** — post list with title, ISO date, and summary; click selects a post.
    - **Right pane** — an Electron `<webview>` whose `src` is set to the selected post's URL. The webview is origin-locked to `https://ryanjosephkamp.github.io` at the React layer; off-origin `new-window` and `will-navigate` events are routed to the user's default browser via `window.resmonAPI.openPath`.
- On feed-fetch failure, the tab falls back to a single-pane embedded view of the blog index plus an **Open in browser** button.
- The Electron main process enables the `<webview>` tag and applies a `will-attach-webview` hardening hook that scrubs `nodeIntegration`, `preload`, and any non-https `src` so the embed cannot reach the host's IPC bridge. The renderer's CSP is widened to allow the GitHub Pages origin for `connect-src` (feed fetch) and `frame-src` / `child-src` (webview embed). See system_info.md for details.
- Blog source of truth lives under `docs/` at the repository root (Jekyll, served from `main` / `docs` via GitHub Pages): `docs/_config.yml`, `docs/index.md`, `docs/_posts/YYYY-MM-DD-<title>.md`. The Atom feed at `feed.xml` is auto-generated by the `jekyll-feed` plugin.

Tab: About App

- Component: `AboutAppTab` (resmon_scripts/frontend/src/components/AboutResmon/AboutAppTab.tsx) — relocated from `Settings → About App` in Update 3.
- Top-level structure: an `<h2>About App</h2>` heading, a `PageHelp` block (`storageKey="about-resmon-about-app"`, title `"About App"`), an `.about-grid` with five `.about-card` sections, and a `.about-footer-note` copyright line.
- The five cards:
    i. **Version** — `resmon` version `<backendVersion>` plus a muted "Current release line: 1.1.x" line.
    ii. **Recent Update** — Update 3 summary (Calendar timezone fix, 12-month horizon expansion with past-horizon notice, AI-API-key deep-link button on Repositories page, new About resmon page with Tutorials / Issues / Blog / About App tabs, per-page tutorial deep-link buttons, shared `RoutineEditModal` with cross-page bus invalidation, Calendar popover enhancements, Configurations View JSON button, executions-to-saved-configuration linkage, and the Settings → Advanced Danger Zone).
    iii. **License** — MIT License summary and permission notice.
    iv. **Privacy Notice** — bullet list calling out local-first execution storage, OS-keychain credential storage, explicit-only outbound calls, and user-controlled exports / emails / cloud uploads.
    v. **Author** — profile photo (bundled `assets/kamp_profile_pic.png`), name (`Ryan Kamp`), role line (`Creator of resmon`), and five social links (GitHub, LinkedIn, X, Website, Email) rendered as `.about-link-btn` buttons with inline SVG icons.

- The footer note prints `Copyright (c) <current year> Ryan Kamp.` using `new Date().getFullYear()` .
- The author social links open in a new tab (`target="_blank"` , `rel="noreferrer"`) except for the `mailto:` link, which is rendered without `target` / `rel` .

Component: TutorialLinkButton

- Component: `TutorialLinkButton` (resmon_scripts/frontend/src/components/AboutResmon/TutorialLinkButton.tsx).
- Props:
  - `anchor: string` — the tutorial-section anchor to deep-link to (matches the corresponding `TutorialSection.anchor`).
  - `label?: string` — optional button label; defaults to `'Tutorial'` .
- Renders a `<button type="button">` with className `btn btn-sm btn-primary tutorial-link-btn` , `data-testid={` tutorial-link-ParseError: KaTeX parse error: Expected 'EOF', got '}' at position 10: {anchor}`}`, and `aria-la…{anchor} tutorial `}` . The click handler calls `navigate({ pathname: '/about-resmon/tutorials', hash: anchor })` .
- Wired into every top-level page header and every Settings sub-panel header. Anchor mapping:
  - Dashboard → `dashboard` ; Deep Dive → `deep-dive` ; Deep Sweep → `deep-sweep` ; Routines → `routines` ; Calendar → `calendar` ; Results & Logs → `results` ; Configurations → `configurations` ; Monitor → `monitor` ; Repositories & API Keys → `repositories` ; Settings → `settings` .
  - Settings sub-panels: Email → `settings-email` ; Cloud Account → `settings-account` ; Cloud Storage → `settings-cloud` ; AI → `settings-ai` ; Storage → `settings-storage` ; Notifications → `settings-notifications` ; Advanced → `settings-advanced` .

Sidebar Wiring

- The left sidebar exposes an **About resmon** entry that routes to `/about-resmon` , alongside the ten existing top-level entries (Dashboard, Deep Dive, Deep Sweep, Routines, Calendar, Results & Logs, Configurations, Monitor, Repositories & API Keys, Settings). The About App entry that previously lived inside Settings has been removed; its content now lives under the About App tab on this page.

Backend

API Endpoints

Method	Path	Purpose	Tab
GET	`/api/health`	Daemon status `{ status, pid, started_at, version }` — used to populate the version banner	About App
GET	`https://ryanjosephkamp.github.io/resmon/feed.xml`	Public GitHub Pages Atom feed of the resmon blog; parsed client-side and rendered in the embedded webview	Blog
(external)	`mailto:ryanjosephkamp@gmail.com?...`	`mailto:` deep link opened via `shell.openExternal` ; not a `resmon` endpoint	Issues
(external)	`https://github.com/ryanjosephkamp/resmon/issues/new?...`	GitHub issue-form deep link opened via `shell.openExternal` ; not a `resmon` endpoint	Issues

The Tutorials tab issues no backend calls; it is fully driven by the static `sections` array compiled into the renderer bundle.

Request and Response Patterns

- `AboutAppTab` issues one `apiClient.get<HealthResponse>('/api/health')` call on mount. Only the `version` field is read; missing or empty fields leave the seeded value (`'1.0.0'`) in place. The fetch is guarded by a `cancelled` flag in the cleanup function so a fast unmount does not write into stale state.
- The shared `apiClient` wrapper resolves the backend port via `window.resmonAPI.getBackendPort()` (with a fallback to the `8742` default) and prefixes the path automatically.

Persistence Touchpoints

- No SQLite, OS-keychain, Drive-OAuth, or APScheduler interactions originate from this page.
- The `PageHelp` block on the About App tab persists its collapsed / expanded state in `localStorage` under the prefix used by `PageHelp` for `storageKey="about-resmon-about-app"` , consistent with every other `PageHelp` instance in the app.

Execution Side Effects

- None. The page neither launches executions nor mutates settings, credentials, configurations, or routines. The Issues tab opens an external URL (`mailto:` or GitHub issues page) via `shell.openExternal` , but `resmon` itself never transmits the report. The Blog tab fetches the public Atom feed and renders embedded post content in a sandboxed `<webview>` . All other outbound effects are in-place URL navigation produced by clicking a TOC entry, a `TutorialLinkButton` , a section's destination button, or a section's prev / next button.

Endpoints: Tutorials Tab

- None — the tab is fully static and renders entirely from the bundled `sections` array.

Endpoints: Issues Tab

- None on the local backend. The tab opens external URLs only — a `mailto:` link for the email path and a `https://github.com/ryanjosephkamp/resmon/issues/new?...` link for the GitHub path — through `window.resmonAPI.openPath` (Electron `shell.openExternal`).

Endpoints: Blog Tab

- `GET https://ryanjosephkamp.github.io/resmon/feed.xml` — Atom feed for the resmon blog (CSP-allowlisted in `index.html`); parsed client-side via `DOMParser` . Each post URL is rendered inside a sandboxed Electron `<webview>` whose origin is locked to `https://ryanjosephkamp.github.io` .

Endpoints: About App Tab

- `GET /api/health` — returns `{ status, pid, started_at, version }` . Only `version` is consumed; the result populates the **Version** card.

Cross-References

- The Tutorials tab's per-section copy is grounded in the matching `*_info.md` document under resmon_reports/info_docs/. Keep the section list in lock-step with those sources; any drift is a documentation bug.
- The About App tab replaces the previous **Settings** → **About App** sub-tab; see settings_info.md for the historical Settings panel listing (which now omits the About App entry).
- The deep-link button surface is rendered on every page; see each page's info document (dashboard_info.md, deep_dive_info.md, deep_sweep_info.md, routines_info.md, calendar_info.md, results_and_logs_info.md, configs_info.md, monitor_info.md, repos_and_api_keys_info.md, settings_info.md) for the per-page header that hosts the corresponding `TutorialLinkButton` .